



INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo



Práctica 7

CONTADOR CON INTERRUPCIONES EXTERNAS

Sistemas en Chips

- ▶ **Prof. José Juan Pérez Pérez**
- ▶ **Grupo: 6CVI**
- ▶ **Equipo I:**
 - ▶ Ayala Fuentes Sunem Gizeht
 - ▶ Guevara Badillo Areli Alejandra
 - ▶ León Pérez May Carolina

▶ viernes, 7 de noviembre de 2025

Introducción

Interrupciones externas

En la arquitectura de computadoras, una interrupción es una señal recibida por el procesador que indica que un evento prioritario requiere atención inmediata. A diferencia del método de sondeo (*polling*), donde el procesador verifica constantemente el estado de un periférico, las interrupciones permiten que el procesador ejecute su tarea principal y solo se detenga cuando el evento ocurre, optimizando el uso de la CPU.

Cuando se dispara una interrupción, el microcontrolador realiza los siguientes pasos automáticos:

1. Termina la instrucción actual.
2. Almacena la dirección de la siguiente instrucción en la **Pila (Stack)**.
3. Salta a una dirección de memoria predefinida llamada **Vector de Interrupción**.
4. Ejecuta la **Rutina de Servicio de Interrupción (ISR)**.
5. Al finalizar (mediante una instrucción de retorno especial), recupera la dirección de la Pila y continúa el programa principal.

Interrupciones Externas en AVR

Las interrupciones externas son aquellas generadas por cambios de estado lógico en pines físicos específicos del microcontrolador. Son fundamentales para detectar eventos del mundo real, como la pulsación de un botón o la señal de un sensor, en tiempo real.

Modos de Disparo (Triggering)

El hardware de interrupción externa puede configurarse para detectar diferentes tipos de estímulos eléctricos:

- **Nivel Bajo:** La interrupción se ejecuta mientras el pin se mantenga en estado lógico bajo (0V).
- **Cambio Lógico:** Se dispara si el pin cambia de estado (de 0 a 1 o de 1 a 0).
- **Flanco de Bajada (Falling Edge):** Se dispara únicamente en la transición de alto a bajo (5V → 0V).
- **Flanco de Subida (Rising Edge):** Se dispara únicamente en la transición de bajo a alto (0V → 5V).

Registros de Control Asociados

Para gestionar estas interrupciones, la arquitectura AVR utiliza tres registros de control principales:

- **Registro de Control de la MCU (MCUCR):** Este registro contiene los bits de configuración que definen el *modo de disparo* (flanco o nivel) para cada interrupción externa. Determina la sensibilidad eléctrica de los pines de entrada.
- **Registro General de Control de Interrupciones (GICR):** Actúa como una "máscara". Permite habilitar o deshabilitar individualmente cada interrupción externa. Aunque ocurra un evento físico, si el bit correspondiente en el GICR no está activo, el microcontrolador ignorará la señal.
- **Registro General de Banderas de Interrupción (GIFR):** Cuando ocurre un evento (ej. un flanco de bajada), el hardware levanta una "bandera" (flag) en este registro para indicar que la petición está pendiente. Si las interrupciones están habilitadas, esta bandera se limpia automáticamente al entrar a la ISR.

El Registro de Estado (SREG) y la Habilitación Global

Existe una jerarquía en el sistema de interrupciones. Además de las habilitaciones individuales (en el GICR), existe un "interruptor maestro" conocido como **Habilitación Global de Interrupciones** (*Global Interrupt Enable*).

Este bit se encuentra en el **Registro de Estado (SREG)**.

- Si este bit es '0', ninguna interrupción puede ocurrir, protegiendo secciones críticas de código.
- Si este bit es '1', el microcontrolador atenderá las interrupciones que estén individualmente activadas.
- Al entrar a una ISR, el hardware pone este bit en '0' automáticamente para evitar interrupciones anidadas infinitas, y lo vuelve a poner en '1' al ejecutar la instrucción de retorno de interrupción.

La Pila (Stack) y el Contexto

La Pila es un área de memoria RAM utilizada para el almacenamiento temporal de datos, fundamental para el manejo de subrutinas e interrupciones. El **Puntero de Pila (Stack Pointer - SP)** es un registro que indica la dirección actual de la cima de la pila.

Cuando ocurre una interrupción, el hardware utiliza la pila para guardar el Contador de Programa (PC), asegurando que el sistema "recuerde" dónde estaba antes de ser interrumpido. Es responsabilidad del programador utilizar la pila dentro de la ISR para guardar el estado de otros registros importantes (contexto) si estos van a ser modificados, garantizando la integridad del programa principal al regresar.

Planteamiento del ejercicio

El objetivo de esta práctica es diseñar e implementar un sistema contador digital de dos dígitos (00-99) utilizando el microcontrolador **ATmega8535**.

El sistema debe cumplir con las siguientes especificaciones funcionales:

1. El sistema debe iniciar en cero. El usuario incrementará la cuenta manualmente pulsando un botón conectado a la interrupción **INT0**.
2. Al presionar un segundo botón conectado a **INT1**, el sistema debe "capturar" el valor actual de la cuenta manual. Inmediatamente después de la captura, el sistema debe reiniciar la cuenta a 00 e iniciar un conteo ascendente automático con una cadencia de 1 segundo por incremento. El conteo debe repetirse cíclicamente desde 00 hasta el Límite capturado.
3. Durante el conteo, el botón de **INT1** cambiará su función para actuar como un control de **Pausa/Reanudar**. Si se presiona, el conteo automático se detiene; si se presiona nuevamente, el conteo continúa.

Desarrollo

Material

Para la realización de esta práctica se requieren los siguientes componentes y herramientas:

- Microcontrolador ATmega8535.
- 2 Displays de 7 Segmentos. Configuración de Cátodo Común.
- 3 Pulsadores (Push-buttons).
- 2 capacitores de 1 μ F a 16V.
- 1 resistencia de 1K Ω .
- Fuente de alimentación: 5V DC.
- 1 placa de prueba.
- Cables de conexión.

Código

```
.include "m8535def.inc"

.def temp = r16
.def unidades = r17
.def decenas = r18
.def limite = r19
.def modo = r20      ; 0 = manual, 1 = automático
.def pausa = r26     ; 0 = corriendo, 1 = pausado

.org 0x0000
rjmp RESET
.org INT0addr
rjmp ISR_INT0
.org INT1addr
rjmp ISR_INT1
.org 0x0020

RESET:
clr r1
ldi temp, LOW(RAMEND)
out SPL, temp
ldi temp, HIGH(RAMEND)
out SPH, temp

; Configuración de Puertos
ldi temp, 0xFF
out DDRA, temp      ; PORTA = Salida (Decenas)
out DDRC, temp      ; PORTC = Salida (Unidades)

; Configuración de Interrupciones (PD2 y PD3)
cbi DDRD, PD2       ; PD2 como entrada
cbi DDRD, PD3       ; PD3 como entrada
sbi PORTD, PD2      ; Pull-up en PD2
sbi PORTD, PD3      ; Pull-up en PD3

; Configuración de Flancos (Bajada) y Habilitación
ldi temp, (1 << ISC01) | (1 << ISC11)
out MCUCR, temp
ldi temp, (1 << INT0) | (1 << INT1)
out GICR, temp

; Inicialización de variables
clr unidades
clr decenas
clr limite
clr modo
clr pausa

sei                ; Habilitación Global de Interrupciones

TABLA7SEG:
.db 0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F
```

```

MAIN:
    rcall MOSTRAR          ; Refresco constante del display
    tst modo               ; Verificar modo (0=Manual, 1=Auto)
    breq MAIN              ; Si es 0, mantener en bucle manual
    rjmp AUTO              ; Si es 1, saltar a rutina automática

AUTO:
    clr unidades
    clr decenas

AUTO_LOOP:
    rcall MOSTRAR          ; Mantiene displays encendidos

PAUSA_CHECK:
    tst pausa              ; Verificar bandera de pausa
    breq CONTAR            ; Si no hay pausa, contar
    rcall MOSTRAR          ; Si hay pausa, refrescar display
    rjmp PAUSA_CHECK       ; y volver a verificar

CONTAR:
    rcall RETARDO_1S       ; Espera de 1 segundo

    ; Lógica de incremento BCD
    inc unidades
    cpi unidades, 10
    brne CONT_AUTO
    clr unidades
    inc decenas

CONT_AUTO:
    ; Cálculo del valor actual (Decenas*10 + Unidades)
    mov temp, decenas
    ldi r24, 10
    mul temp, r24
    add r0, unidades
    mov r25, r0
    clr r1

    ; Comparación con el Límite
    cp r25, limite
    breq AUTO_LOOP        ; Si llegó al límite, reiniciar cuenta
    brlo AUTO_LOOP        ; Si es menor, seguir bucle
    rjmp AUTO

MOSTRAR:
    mov temp, decenas
    rcall TABLA
    out PORTA, temp        ; Muestra Decenas en Puerto A
    rcall RETARDO_CORTO
    mov temp, unidades
    rcall TABLA
    out PORTC, temp        ; Muestra Unidades en Puerto C
    rcall RETARDO_CORTO
    ret

TABLA:
    push ZH
    push ZL
    ldi ZH, HIGH(TABLA7SEG*2)
    ldi ZL, LOW(TABLA7SEG*2)

```

```

    add ZL, temp
    adc ZH, r1
    lpm temp, Z
    pop ZL
    pop ZH
    ret

; --- Subrutinas de Retardo (Software Loops) ---
RETARDO_CORTO:
    ldi temp, 80
RC1: dec temp
    brne RC1
    ret

RETARDO_1S:
    ldi r21, 10
RET1S_LOOP:
    rcall RETARDO_100MS
    dec r21
    brne RET1S_LOOP
    ret

RETARDO_100MS:
    ldi r22, 100
RET100_LOOP1:
    ldi r23, 250
RET100_LOOP2:
    dec r23
    brne RET100_LOOP2
    dec r22
    brne RET100_LOOP1
    ret

; --- Rutinas de Servicio de Interrupción (ISR) ---
ISR_INT0:                                ; Incremento Manual
    tst modo
    brne FIN_INT0                        ; Si está en Auto, ignorar botón
    inc unidades
    cpi unidades, 10
    brne FIN_INT0
    clr unidades
    inc decenas
    cpi decenas, 10
    brne FIN_INT0
    clr decenas
FIN_INT0:
    reti

ISR_INT1:                                ; Control de Modo y Pausa
    tst modo
    breq INICIAR_AUTO                    ; Si es Manual, cambiar a Auto

    ; Lógica de Pausa (Toggle)
    tst pausa
    breq PON_PAUSA
    clr pausa
    rjmp FIN_INT1

```

```

PON_PAUSA:
    ldi pausa, 1
    rjmp FIN_INT1

INICIAR_AUTO:                ; Captura de Límite y cambio de modo
    mov temp, decenas
    ldi r24, 10
    mul temp, r24
    add r0, unidades
    mov limite, r0            ; Guarda (Decenas*10 + Unidades) en Limite
    clr r1
    ldi modo, 1              ; Cambia bandera a modo Automático
    clr pausa

FIN_INT1:
    reti

```

Ilustración I. Código implementado.

Solución

La solución al problema planteado se basa en una máquina de estados finitos implícita controlada por las variables modo y pausa.

1. **Control de Modos (INT1):** La interrupción **INT1** actúa como el cerebro de transición.
 - Si modo = 0 (Manual): Calcula el valor total actual ($\text{Decenas} * 10 + \text{Unidades}$) usando la instrucción de multiplicación `mul`, almacena este resultado en el registro `limite` y cambia modo a 1.
 - Si modo = 1 (Auto): Alterna el valor del registro `pausa` entre 0 y 1.
2. **Modo Manual (INT0):** La interrupción **INT0** gestiona el incremento. Se implementó una validación crítica: `tst modo`. Si el sistema ya está en modo automático, cualquier pulsación en **INT0** es ignorada, evitando corrupción de la cuenta automática.
3. **Visualización Dedicada:** Se utilizan dos puertos completos (**PORTA** y **PORTC**) para manejar los displays. Aunque esto consume más recursos de E/S que la multiplexación, simplifica la lógica de visualización en la rutina **MOSTRAR**, la cual convierte los valores BCD a patrones de 7 segmentos usando la subrutina **TABLA** y la instrucción **LPM** (Load Program Memory).
4. **Temporización:** Para el modo automático, se generó un retardo de 1 segundo (**RETARDO_1S**) mediante bucles anidados de decremento, asegurando que el conteo sea visible para el usuario.

Circuito

La interfaz de hardware se establece de la siguiente manera:

Visualización de Decenas: Los segmentos (a-g) del primer display se conectan al **Puerto A** (PA0-PA6).

Visualización de Unidades: Los segmentos (a-g) del segundo display se conectan al **Puerto C** (PC0-PC6).

Entradas de Control:

- Pulsador de "Incremento Manual" conectado al pin **PD2 (INT0)**.
- Pulsador de "Fijar Límite / Pausa" conectado al pin **PD3 (INT1)**.
- Ambos pulsadores cierran el circuito a Tierra (GND), aprovechando las resistencias *pull-up* internas activadas en el código.

Diagrama del circuito

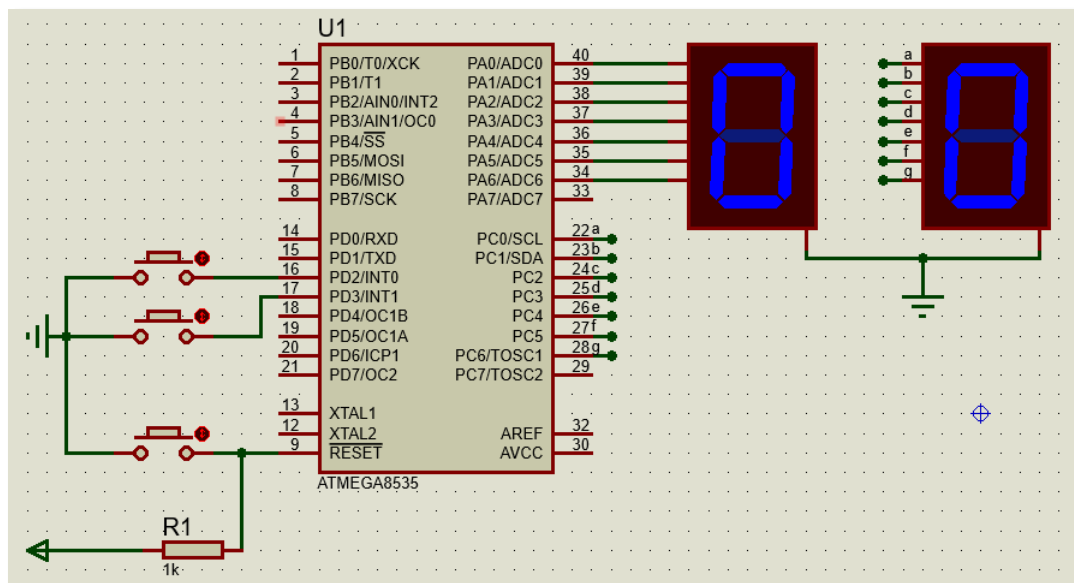


Ilustración 2. Esquema del circuito en proteus.

Simulación

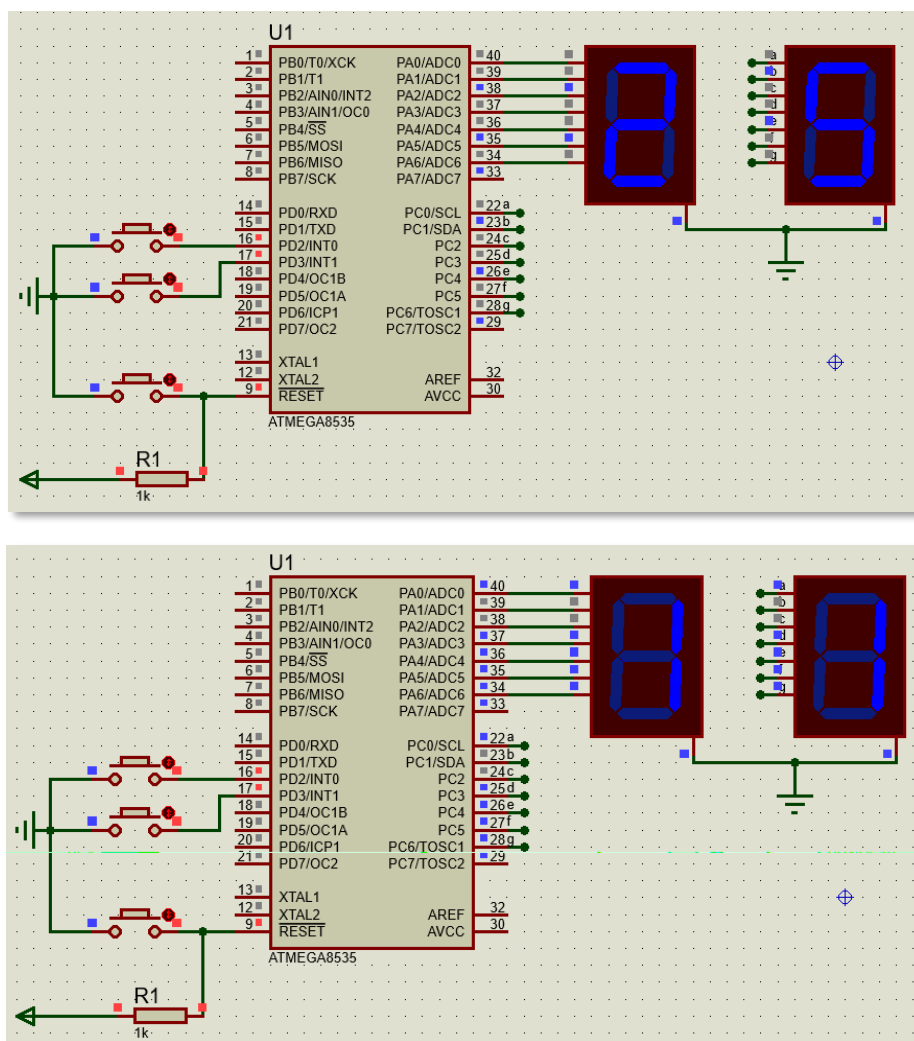


Ilustración 3. Simulación del funcionamiento en proteus.

Circuito Armado

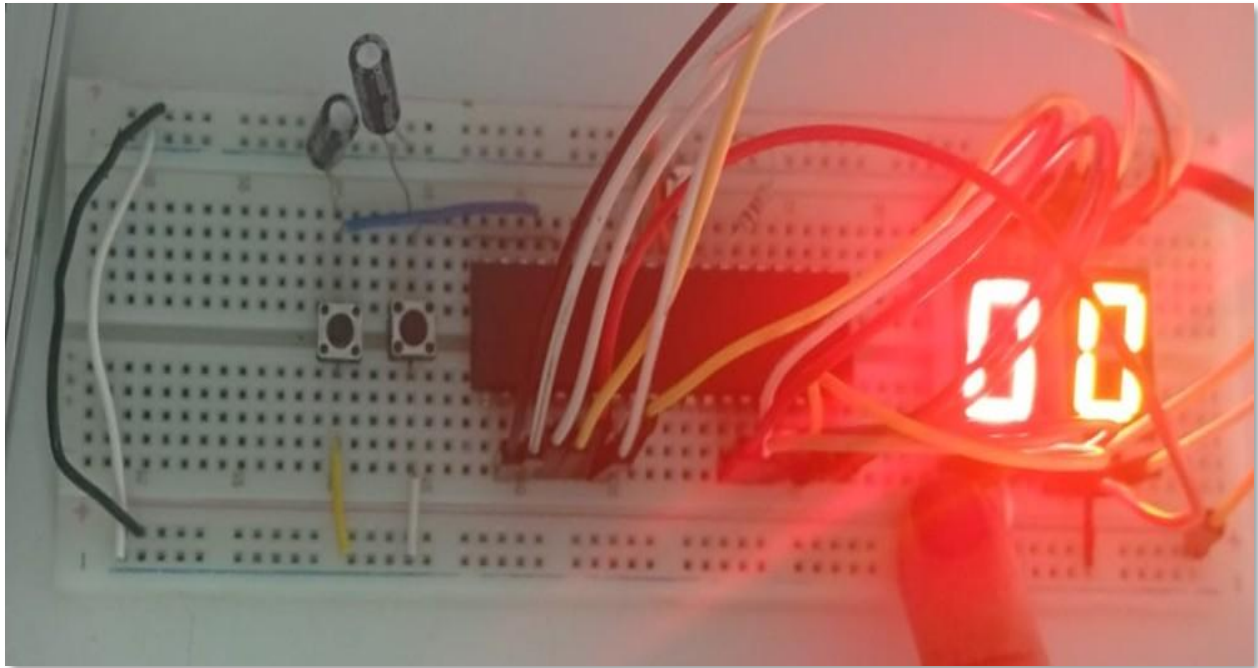


Ilustración 4. Armado físico del circuito funcionando.

Conclusiones

Ayala Fuentes Sunem Gizeht

En esta práctica comprendí que lo que vemos en los displays (Unidades y Decenas separadas) es diferente a cómo el microcontrolador procesa los datos internamente. Para lograr que el contador automático se detuviera exactamente en el número que elegimos manualmente, tuvimos que implementar una **conversión aritmética**: multiplicar las decenas por 10 y sumarle las unidades. Esto me enseñó que en lenguaje ensamblador no basta con guardar los datos para mostrarlos; a veces debemos procesarlos matemáticamente (usando instrucciones como MUL) para poder realizar comparaciones lógicas válidas, como la del límite de conteo.

Guevara Badillo Areli Alejandra

Lo más importante que aprendí fue cómo un mismo botón físico (conectado a la interrupción **INT1**) puede tener comportamientos completamente diferentes dependiendo del 'estado' del programa. Al usar una variable de control (modo), logramos que al principio el botón sirviera para 'Guardar y Arrancar', y después se transformara en un botón de 'Pausa/Reanudar'. Esto concluye que las interrupciones no son rígidas; su función puede ser dinámica si programamos una lógica de estados adecuada, permitiendo interfaces de usuario complejas con pocos botones.

León Pérez May Carolina

Al comparar esta práctica con la anterior, noté la gran diferencia entre usar **Multiplexación** y usar **Puertos Dedicados**. En este ejercicio, al conectar cada display a un puerto completo (Puerto A y Puerto C), el código para mostrar los números fue mucho más sencillo y no requirió temporizadores complejos para el refresco. Mi conclusión es que, como ingenieros, siempre debemos evaluar un compromiso: si queremos ahorrar pines del microcontrolador, el software se vuelve más difícil (multiplexado); pero si queremos simplificar el software, debemos gastar más recursos de hardware (puertos dedicados).

Referencias

- ▶ Circuit Digest. (2018, 20 de agosto). *Interfacing 7 segment display with AVR ATmega32*. Recuperado de <https://circuitdigest.com/microcontroller-projects/interfacing-7-segment-display-with-avr-atmega32>
- ▶ Electronics Hub. (2015, 30 de agosto). *Interfacing 7 segment display with AVR microcontroller*. Recuperado de <https://www.electronicwings.com/avr-atmega/7-segment-display-interfacing-with-atmega16-32>
- ▶ Engineers Garage. (s.f.). *AVR external interrupt tutorial*. Recuperado de <https://www.engineersgarage.com/avr-external-interrupt-tutorial/>
- ▶ MaxEmbedded. (2011, 2 de agosto). *AVR interrupts: External and internal*. Recuperado de <https://maxembedded.com/2011/08/avr-interrupts/>
- ▶ Microchip Developer. (s.f.). *8-bit AVR interrupts*. Recuperado de <https://developerhelp.microchip.com/xwiki/bin/view/products/mcu-mpu/8-bit-avr/peripherals/interrupts/>
- ▶ Tutorialspoint. (s.f.). *AVR microcontroller - Interrupts*. Recuperado de <https://es.scribd.com/document/346420486/Avr-Tutorials-com-AVR-Microcontroller-Interrupts>
- ▶ Extreme Electronics. (s.f.). *Using external interrupts in AVR microcontrollers*. Recuperado de <https://www.electronicwings.com/avr-atmega/atmega1632-external-hardware-interrupts>
- ▶ Atmel Corporation. (2016). *AVR instruction set manual*. Recuperado de <https://www1.microchip.com/downloads/en/DeviceDoc/AVR-InstructionSet-Manual-DS40002198.pdf>